

Kapitel 8

Strukturierung von Programmen

8.1 Gültigkeitsbereiche von Namen

Wo in einem Programm dürfen Vereinbarungen stehen? An verschiedenen Stellen haben wir dazu bereits Details kennengelernt:

- Am Anfang jedes Blocks dürfen Variablen vereinbart werden.
- Funktionen müssen auf gleicher Ebene (wie die Funktion `main`) „global“ definiert werden.
- Benannte Konstanten (mit `#define`) wurden jeweils vor die erste Funktion (das Hauptprogramm) geschrieben.
- Prototypen wurden ebenfalls vor die erste Funktion geschrieben.

Es gibt aber durchaus weitere Möglichkeiten:

- Variablen kann man auch außerhalb von Funktionen vereinbaren.
- Prototypen können auch am Anfang eines Blocks angegeben werden.

Als Beispiel soll das Programm zur Invertierung eines Strings in modifizierter Form dienen:

```
#include <stdio.h>

#define LAENGE 20

/*= Rahmenprogramm =====*/
int main(void) {
    char string[LAENGE + 1];
    int anzahl;
    void invert(char str[]);

    printf("String eingeben (max. %d Zeichen):\n", LAENGE);
    scanf("%s", string);
    invert(string);
    printf("Der invertierte String ist '%s'\n", string);

    return 0;
}

/*= Funktionen =====*/

int i, j;
```

```

/*- Invertieren eines String -----*/
void invert(char str[]) {
    char c;
    int strlen(char str[]);

    for (i = 0, j = strlen (str) - 1; i < j; i++, j--)
        c = str[i], str[i] = str[j], str[j] = c;
}

/*- Bestimmung der Laenge eines String -----*/
int strlen(char str[]) {
    for (i = 0; str[i] != '\0'; i++)
        ;

    return i;
}

```

Quelltext 8.1: Invertieren eines Strings mit Funktionen

Drei wesentliche Änderungen gegenüber der ursprünglichen Lösung sind hervorzuheben:

- Der Prototyp der Funktion **strlen** wurde in die Funktion **invert** hineingezogen.
- Der Prototyp der Funktion **invert** wurde in die Funktion **main** hineingezogen.
- Die Vereinbarungen der **int**-Variablen aus beiden Funktionen wurde herausgezogen und zu einer einzigen Vereinbarung zusammengezogen.

Welche Konsequenzen hat das? Wir sehen uns das im Rahmen der allgemeinen Regeln an:

1. Namen, die innerhalb eines Blocks (also zum Beispiel im Rumpf einer Funktion) deklariert werden, sind nur innerhalb dieses Blocks bekannt; Namen, die außerhalb von Funktionen deklariert werden, sind von der Stelle ihrer Deklaration bis zum Ende der Quelldatei bekannt.

Für unser Beispiel hat das zunächst die Konsequenz, dass die Funktion **strlen** nur noch innerhalb der Funktion **invert** bekannt ist. Das Hauptprogramm könnte sie, wenn es wollte, jetzt nicht mehr aufrufen, was in der ursprünglichen Formulierung ohne weiteres möglich gewesen wäre.

Die Umsetzung des Prototyps von **invert** hat keine Folgen: Die Definition einer Funktion impliziert bei Bedarf deren Deklaration.

Die weitere Konsequenz: Die **int**-Variablen **i** und **j** sind jetzt beiden Funktionen bekannt. Nun könnte man zwar sagen: Was macht es aus, dass die Funktion **strlen** eine Variable (**j**) kennt, die sie nicht braucht. Das ist auch richtig – der eigentliche Unterschied liegt im Bedeutungswandel des Namen **i**: In der ursprünglichen Formulierung hatten beide Funktionen eine Variable **i**. Und diese Variablen hatten außer dem Namen nichts gemeinsam – insbesondere bezeichneten sie verschiedene Speicherplätze. Jetzt teilen sich die beiden Funktionen eine Variable, die natürlich auch nur einen Speicherplatz besitzt – mit der Folge, dass das Programm so nicht funktioniert.

Fazit: Bei Prototypen spricht viel dafür, sie gleich an den Anfang eines Programms zu schreiben, noch vor die erste Funktion, damit man sie bei Bedarf an beliebiger Stelle des Programms verwenden kann. Wenn man Funktionen „verstecken“ will, um ihre Verwendung auf ganz bestimmte Stellen zu beschränken, sollte man das Programm modularisieren – darauf werden wir gleich noch zurückkommen. Umgekehrt spricht

bei Variablen viel dafür, sie möglichst lokal zu deklarieren, in der Regel also innerhalb einer Funktion.

2. Die Namen der Parameter in Prototypen und die Namen der Parameter in einer Funktionsdefinition sind nur innerhalb der Funktion bekannt.

Obwohl sowohl der Parameter von `invert` als auch der Parameter von `strlen` mit `str` bezeichnet wurde, ergibt sich daraus nicht zwangsläufig, dass beide dasselbe Objekt bezeichnen, auch wenn es im Beispiel zufällig so ist.

3. Namen können *verschattet* werden, d.h. ein Name kann vorübergehend eine andere Bedeutung bekommen.

Genutzt haben wir das beim Prototyp von `strlen`: Der Name `str` bezeichnet innerhalb des Prototyps nicht den Parameter von `invert`, wie in den Zeilen davor und dahinter, sondern den Parameter von `strlen`.

In gleicher Weise verschattet jede Deklaration eines Namens innerhalb eines Blocks gleichnamige Größen außerhalb dieses Blocks. Wir könnten zum Beispiel programmieren

```
int i;

for (i = 0; i < laenge; i++) {
    float i;
    /* ... */
}
```

Dass so etwas kaum zweckmäßig ist, dürfte auf der Hand liegen.

Die Regeln, die wir eben besprochen haben, gelten übrigens **nicht** für Konstanten, die mit `#define` benannt werden. Der Hintergrund ist einleuchtend: Der Präprozessor, der `define`-Direktiven verarbeitet, weiß nichts von C – er führt nur formal Zeichenersetzungen aus. Entsprechend ersetzt er von der `define`-Direktive ab jedes Vorkommen des Namens. Der Compiler muss dagegen die Struktur des Programms analysieren – und berücksichtigen.

8.2 Interne und externe Größen

Eine Größe heißt *intern*, wenn ihre Definition innerhalb einer Funktion steht, sonst *extern*.

Funktionen sind offenbar immer extern, da C die Schachtelung von Funktionen nicht erlaubt. Bei Variablen hat man, wie gesehen, die Wahl: Interne Variablen stehen nur innerhalb des Blocks zur Verfügung, der ihre Definition enthält, während externe Variablen allen Funktionen zur Verfügung stehen, die hinter der Variablendefinition definiert werden – es sei denn, die Funktionen verschatten sie.

In der Regel sollte man versuchen, nur mit internen Variablen auszukommen. Es gibt aber durchaus auch Fälle, in denen externe Variablen zweckmäßig sind.

Als Beispiel betrachten wir Funktionen zur Stapelverwaltung (Stack, *lifo* = *last in first out*). Für einen Stapel gibt es vier „Standardoperationen“ in zwei Paaren:

- Bevor man ein Element auf den Stapel legen kann (`push`), muss man prüfen, ob noch Platz auf dem Stapel ist (`full`).
- Bevor man das oberste Element vom Stapel holen kann (`pop`), muss man prüfen, ob überhaupt ein Element auf dem Stapel liegt (`empty`).

Wie die Funktionen den Stapel verwalten, braucht und sollte den Benutzer der Funktionen nicht interessieren. Wir werden den Stapel durch einen Vektor realisieren; denkbar wäre

aber auch eine verkettete Liste. Neben dem Vektor selbst (**stapel**) brauchen wir eine Indexvariable (**spitze**), die die erste freie Komponente des Vektors bezeichnet.

Da alle vier Funktionen auf Vektor und/oder Indexvariable operieren sollen, haben wir zwei Möglichkeiten:

- Wir können Vektor und/oder Indexvariable als Parameter vorsehen. Das hat zur Folge, dass der Benutzer beide deklarieren und immer wieder als Argumente übergeben muss. (Die maximale Höhe des Stapels sollte bzw. müsste dann ebenfalls als Parameter übergeben werden.)
- Wir können Vektor und Indexvariable als externe Variablen vereinbaren. Sie brauchen dann nicht mehr als Argumente übergeben zu werden. Und wir halten uns gleich noch eine Möglichkeit offen: Wir können die Funktionen jederzeit von der Vektorspeicherung auf eine verkettete Liste umstellen, ohne dass irgendwelche Änderungen an den Aufrufen vorzunehmen sind.

Für die konkrete Realisierung soll angenommen werden, dass Zeichen zu speichern sind. Ein Rahmenprogramm soll dem Benutzer die Möglichkeit geben, ein Zeichen auf den Stapel zu legen oder herunterzunehmen bzw. das Programm zu beenden.

```
#include <stdio.h>

/** Prototypen *****/
int full(void);
int empty(void);
void push(char zeichen);
char pop(void);

char gelesen(void);

/*= Rahmenprogramm =====*/
int main(void) {
    do {
        printf("0 = Zeichen auf Stapel ablegen\n");
        printf("1 = Zeichen von Stapel nehmen\n");
        printf("2 = Programmende\n");
        switch (gelesen()) {
            case '0':
                if (full ())
                    printf("Stapel ist voll!\n");
                else {
                    printf("Zeichen eingeben: ");
                    push(gelesen ());
                }
                break;
            case '1':
                if (empty())
                    printf("Stapel ist leer!\n");
                else
                    printf("Oberstes Zeichen ist: %c\n", pop());
                break;
            case '2':
                return 0;
                break;
            default:
                printf("Nur 0, 1 oder 2 ist erlaubt!\n");
        }
    }
}
```

```
    } while (1);
}

/*- Lesen eines Zeichen (Hilfsroutine) -----*/
char gelesen(void) {
    char c1, c2;
    c1 = c2 = getchar();
    while (c2 != '\n')
        c2 = getchar();
    return c1;
}

/*= Stapel-Funktionen mit Daten =====*/

#define HOEHE 10

char stapel[HOEHE];
int spitze = 0;

int full(void) {
    return spitze >= HOEHE;
}

int empty(void) {
    return spitze <= 0;
}

void push(char zeichen) {
    stapel[spitze++] = zeichen;
}

char pop(void) {
    return stapel[--spitze];
}
```

Quelltext 8.2: Stapelverwaltung

Sind externe Variablen gelegentlich auch sehr nützlich, so sollte man sich doch von Fall zu Fall sehr genau überlegen, ob man sie verwendet oder nicht. Bei extensiver Verwendung externer Variablen geht sehr schnell der Überblick verloren, wann wo was verändert wird – entsprechend sind nachträgliche Änderungen an den Funktionen extrem gefährlich.

8.3 Das Modulkonzept

Das letzte Beispiel legt mit seiner geschilderten Intention nahe, dass man das nutzende Programm und die Funktionen zur Stapelverwaltung nicht nur optisch innerhalb einer Quelldatei voneinander trennt, sondern dass man sie in verschiedene Quelldateien schreibt. So, wie das Beispiel vorab beschrieben wurde, soll die Stapelverwaltung ja beliebigen Programmen zur Verfügung stehen; unser Hauptprogramm muss man auch eher als „Testrahmen“ bezeichnen – eine Stapelverwaltung als Selbstzweck wie im Beispiel ist natürlich ziemlich unsinnig.

Eine Zerlegung in zwei Quelldateien ist auch mit den uns bekannten Mitteln bereits möglich: Wenn wir die Funktionen zur Stapelverwaltung in eine separate Quelldatei schreiben, etwa `stapel.c`, können wir diese Datei mit

```
#include "stapel.c"
```

ohne weiteres in beliebige andere Dateien einfügen. (Die Gänsefüßchen bewirken, dass der Präprozessor an anderen Stellen im Dateibaum sucht, als wenn spitze Klammern verwendet werden.)

Dieses Vorgehen erweist sich bei der praktischen Arbeit an größeren Projekten schnell als unzureichend. Was man braucht, ist eine Möglichkeit, Teile eines Programms voneinander unabhängig zu übersetzen und erst mit dem Linker zusammenzufügen. Solche Teile, die unabhängig voneinander übersetzt werden können, werden *Module* genannt. C geht sogar noch einen Schritt weiter: Ein Modul, der nur Unterprogramme enthält, wird in zwei Quelldateien gespeichert:

- Eine Datei enthält nur die Deklarationen der Größen, die der Modul *exportiert*, also in der Regel Typdeklarationen und Prototypen von Funktionen, gelegentlich auch Konstanten und Variablen. Solch eine Datei wird als *Headerdatei* bezeichnet.
- Die andere Datei enthält die Definitionen der zu exportierenden Größen, also insbesondere die Definitionen der Prototypen aus der Headerdatei. Sie kann aber auch zusätzliche lokale Funktionen realisieren, die dann nur innerhalb des Moduls verwendet werden können. Auf die Headerdatei greift sie mit einer `include`-Direktive zu.

Üblicherweise erhalten Headerdatei und Implementation eines Moduls den selben Namen, bei der Headerdatei um `.h` und bei der Implementation um `.c` erweitert.

Will man auf einen separaten Modul zugreifen, muss man zwei Dinge tun:

- Dem Compiler muss man im *rufenden* Modul die Größen des *gerufenen* Moduls bekanntmachen. Das geschieht mit einer `include`-Direktive für die Headerdatei desgerufen Moduls.
- Dem Linker müssen die Module, die er zusammenbinden soll, einzeln genannt werden.

Sehen wir uns zunächst die Zerlegung unseres Programmbeispiels an: Die Headerdatei zur Stapelverwaltung, die `stapel.h` heißen soll, kann so aussehen:

```
/** Prototypen *****/
int full (void);
int empty (void);
void push (char zeichen);
char pop (void);
```

Quelltext 8.3: Stapelverwaltung (modularisiert I): `stapel.h`

Dazu gehört die Implementation `stapel.c`:

```
#include "stapel.h"

/*= Daten =====*/
#define HOEHE 10

char stapel[HOEHE];
int spitze = 0;

/*= Funktionen =====*/
int full (void) {
    return spitze >= HOEHE;
}
```

```

int empty (void) {
    return spitze <= 0;
}

void push (char zeichen) {
    stapel[spitze++] = zeichen;
}

char pop (void) {
    return stapel[--spitze];
}

```

Quelltext 8.4: Stapelverwaltung (modularisiert I): `stapel.c`

Schließlich haben wir den *Hauptmodul*, d.h. den Modul, der das Hauptprogramm enthält. Sein Name sei `stapeltest.c`.

```

#include <stdio.h>
#include "stapel.h"

/** Prototypen *****/
char gelesen( void);

/*= Rahmenprogramm =====*/
int main (void)
{
    do {
        printf ("0 = Zeichen auf Stapel ablegen\n"
               "1 = Zeichen von Stapel nehmen\n"
               "2 = Programmende\nEingabe: ");
        switch (gelesen ()) {
            case '0':
                if (full ())
                    printf ("Stapel ist voll!\n");
                else {
                    printf ("Zeichen eingeben: ");
                    push (gelesen ());
                }
                break;
            case '1':
                if (empty ())
                    printf ("Stapel ist leer!\n");
                else
                    printf ("Oberstes Zeichen ist: %c\n", pop());
                break;
            case '2':
                return 0;
                break;
            default:
                printf ("Nur 0, 1 oder 2 ist erlaubt!\n");
        }
    } while (1);
}

/*- Lesen eines Zeichen (Hilfsroutine) -----*/
char gelesen (void) {

```

```
int c1, c2;
c1 = c2 = getchar ();
while (c2 != '\n')
    c2 = getchar ();
return (char) c1;
}
```

Quelltext 8.5: Stapelverwaltung (modularisiert I): `stapeltest.c`

Da ein Hauptmodul nichts exportiert, gibt es zu ihm auch keine Headerdatei.

8.4 Separate Compilation, make

Wir müssen uns jetzt noch ansehen, wie man Programme, die aus mehreren Modulen bestehen, übersetzen und binden kann. Letzlich ist das ganz einfach. Für unser Beispielprogramm können wir etwa schreiben

```
gcc -Wall -ansi stapeltest.c stapel.c
```

wenn wir mit `a.out` als Namen für das ausführbare Programm zufrieden sind, bzw.

```
gcc -Wall -ansi -o stapeltest stapeltest.c stapel.c
```

wenn das ausführbare Programm den Namen `stapeltest` erhalten soll.

Konnten beide Module nacheinander fehlerfrei übersetzt werden, schließt sich der Lauf des Linkers unmittelbar an, so dass wir das ausführbare Programm erhalten. Stellt der Compiler dagegen einen Fehler fest, etwa im Modul `stapeltest.c`, so übersetzt er zwar auch noch `stapel.c` (oder versucht es zumindest); der Linker wird jedoch nicht mehr gestartet.

Der nächste Schritt ist nun natürlich, die Fehler in `stapeltest.c` zu beseitigen. Danach kann man dann neu übersetzen.

Für größere Projekte ist dieses Vorgehen offensichtlich ungeeignet: Wer mag schon bei jedem Compileraufruf zehn oder mehr Module hinschreiben, ganz davon abgesehen, dass es bei so vielen Modulen nicht besonders effizient ist, alle immer neu zu übersetzen. Günstiger ist es, die Objektdateien aufzubewahren und nur dann neu zu erzeugen, wenn sie nicht mehr auf dem aktuellen Stand sind. Dafür gibt es zwei Gründe:

- Offensichtlich muss ein Modul neu übersetzt werden, wenn man an seiner Implementation etwas geändert hat.
- Bei etwas näherem Hinsehen ist es ebenso offensichtlich, dass ein Modul neu übersetzt werden muss oder zumindest der Sicherheit halber neu übersetzt werden sollte, wenn an einer der Headerdateien, die er benutzt, eine Änderung vorgenommen wurde: Es kann ja zum Beispiel die Parameterliste einer Funktion verändert worden sein. Statt sich nun den Kopf darüber zu zerbrechen, wo die Funktion überall aufgerufen wird, lässt man den Compiler arbeiten. Er wird (wenn er dem Standard entspricht) jede Abweichung der Argumente eines Aufrufs von der (geänderten) Parameterliste „bemeckern“!

UNIX bietet für solche Fälle eine Hilfe an, nämlich das Programm `make`. Für eine detaillierte Besprechung fehlt leider die Zeit. Aber exemplarisch soll das Programm wegen seiner Bedeutung doch kurz vorgestellt werden:

Im ersten, nur einmal auszuführenden Schritt, erstellt man mit einem Texteditor eine *Beschreibungsdatei*, d.h. man schreibt auf, was zu tun ist, um ein Programm auf den neuesten Stand zu bringen. Für unser Beispiel könnte das so aussehen:


```
# GCC Compiler-Optionen
GCCFLAGS = -ansi -pedantic -Wall

# Abhaengigkeiten und Erzeugungskommandos
stapeltest : stapeltest.o stapel.o
            gcc -o stapeltest stapeltest.o stapel.o

stapeltest.o : stapeltest.c stapel.h
            gcc $(GCCFLAGS) -c stapeltest.c

stapel.o : stapel.c stapel.h
            gcc $(GCCFLAGS) -c stapel.c
```

Quelltext 8.6: makefile für Stapelverwaltung

Gibt man dieser Datei den Namen `makefile`, so kann man ihre Ausführung durch das Kommando `make` starten.

Die Zeilen der Beschreibungsdatei sind so zu interpretieren:

- Die erste Zeile ist eine Kommentarseile.
- Die Zeile

```
stapeltest : stapeltest.o stapel.o
```

bedeutet: Wenn die Datei `stapeltest` älter als eine der beiden nachfolgend angegebenen Objektdateien ist, werden das oder die nachfolgenden Kommandos ausgeführt, sonst übersprungen.

- Die nächste Zeile

```
gcc -o stapeltest stapeltest.o stapel.o
```

ist ein solches Kommando: Die Objektdateien werden gebunden. Solche Kommandozeilen müssen mit einem (horizontalen) Tabulator beginnen, während die Beschreibungen der Abhängigkeiten jeweils am Zeilenanfang beginnen müssen.

- Bevor `make` das Alter von `stapeltest`, `stapeltest.o` und `stapel.o` vergleicht, prüft es zunächst, ob weiter unten Regeln angegeben sind, wie ggf. `stapel.o` und `stapeltest.o` auf den neuesten Stand zu bringen sind. Diese Regeln folgen jetzt also, mit der gleichen Syntax wie die „Hauptregel“ am Anfang. Anzumerken ist nur noch: Die Option `-c` bewirkt, dass nur eine Objektdatei erzeugt, nicht jedoch der Linker aufgerufen wird. Das Erstellen des Programms wird hier also tatsächlich in die Teilaufgaben übersetzen und binden zerlegt.

Die Option `-Wall` im ersten `gcc`-Aufruf wurde übrigens nicht vergessen: Er bewirkt ja keine Übersetzung von Quellcode, sondern nur noch das Binden der Objektdateien.

8.5 Lokale und globale Größen

Das Beispiel zur modularisierten Lösung der Stapelverwaltung ist noch nicht ganz fertig. Es tut nämlich noch nicht wirklich das, was beschrieben wurde: Die Variablen `stapel` und `spitze` sind *noch nicht* im Modul „versteckt“!

Der Hintergrund ist, dass bei der Beschreibung der Gültigkeitsbereiche von Namen das Modulkonzept noch völlig außer Acht gelassen wurde. Jede externe Größe eines Moduls

ist, wenn man nicht ausdrücklich anderes bestimmt, gleichzeitig eine *globale* Größe, d.h. man kann auf sie auch aus anderen Modulen heraus zugreifen – ob sie in der zugehörigen Headerdatei deklariert wird oder nicht, ist dafür gleichgültig. Interne Variablen sind dagegen gleichzeitig stets *lokale* Größen, d.h. man kann auf sie nur innerhalb der Funktion zugreifen, in der ihre Vereinbarung steht. Ebenso sind benannte Konstanten, die in einer Implementation in einer **define**-Direktive definiert werden, stets lokale Größen. Dass eine benannte Konstante, die in einer Headerdatei in einer **define**-Direktive definiert wird, allen Modulen zur Verfügung steht, die die Headerdatei verwenden, sollte offensichtlich sein.

Es gibt jedoch eine einfache Möglichkeit, für externe Größen zu verhindern, dass sie gleichzeitig auch globale Größen sind: Man fügt der Definition der Größe das Speicherklassen-Attribut **static** hinzu. Es ist üblich, wenn auch nicht notwendig, dieses Attribut an den Anfang der Definition setzen.

Entsprechend sollte unsere Datei **stapel.c** so beginnen, damit die Variablen **stapel** und **spitze** im Modul „versteckt“ werden:

```
#include "stapel.h"

/*= Daten =====*/
#define HOEHE 10

static char stapel[HOEHE];      /* Die Daten sind jetzt */
static spitze = 0;              /* im Modul versteckt */

...
```

Quelltext 8.7: Stapelverwaltung (modularisiert II): **stapel.c**

In diesem Beispiel sind es zwei Variablen, denen wir das Attribut **static** gegeben haben. Ebenso kann man aber auch (Hilfs-)Funktionen damit versehen, wenn diese Funktionen nur innerhalb des Moduls aufgerufen werden können bzw. sollen. Ein Beispiel ist die Funktion **gelesen** im (Test-)Hauptmodul der Stapelverwaltung. Definieren wir sie als **static**, so ist sie im Modul „versteckt“; verzichten wir dagegen auf das Attribut **static**, so wird der Name **gelesen** an den Linker weitergegeben – und der meckert, falls in irgendeinem anderen der einzubindenden Module auch eine globale Variable oder Funktion mit dem Namen **gelesen** definiert ist.

Fazit: Ebenso, wie man Variablen möglichst als interne Größen einer Funktion vereinbaren sollte, sollte man darauf achten, dass in jedem Modul alle Größen als lokal vereinbart werden, die nur innerhalb des Moduls (sinnvoll) verwendet werden können.

Man hat letztlich zwei voneinander unabhängige Ebenen des „Versteckens“, nämlich einmal die Ebene des Übersetzens und einmal die Ebene des Bindens.

8.6 Deklarationen und Definitionen

Bei den Funktionen haben wir bereits gesehen, dass man zwischen Deklaration und Definition zu unterscheiden hat. Mit dem Modulkonzept haben wir jetzt auch gesehen, warum diese Unterscheidung erforderlich ist: Die Deklaration einer Funktion erlaubt den Zugriff auf sie, d.h. sie erlaubt den Aufruf der Funktion und damit die Ausführung ihrer Operationen; sie besagt aber nichts darüber, welche Operationen ihr Aufruf bewirkt. Die Definition einer Funktion legt ihre Operationen im Fall des Aufrufes fest, bewirkt aber nicht ihre Ausführung.

Es ist offensichtlich, dass eine Funktion innerhalb genau eines Moduls eines Gesamtprogramms *definiert* werden darf und muss, während sie in beliebig vielen Modulen, ggf. auch keinem, *deklariert* werden darf.

Wir haben bereits gesehen, dass auch Variablen global oder lokal sein können. Entsprechend muss es auch für Variablen die Unterscheidung zwischen Deklaration und Definition geben. Hierzu dient das Speicherklassen-Attribut **extern**: Eine Vereinbarung, die es enthält, ist eine Deklaration, und eine Vereinbarung, die es nicht enthält, eine Definition.

Sehen wir uns ein etwas konstruiertes Beispiel an – es dient nur der Demonstration des Konzepts. Angenommen man müsste – aus welchen Gründen auch immer – darauf verzichten, eine separate Funktion **empty** für die Stapelverwaltung zu definieren. Dann könnte man wieder auf den ursprünglichen Einsatz der Variablen **spitze** zurückgreifen: Der Ausdruck **!spitze** leistet ja im Grunde dasselbe wie der Aufruf der Funktion **empty**.

```
/** Prototypen *****/
int full (void);
void push (char zeichen);
char pop (void);
extern int spitze;          /* auf spitze kann jetzt aus */
                           /* beliebigen Modulen */
                           /* zugegriffen werden */
```

Quelltext 8.8: Stapelverwaltung (modularisiert III): **stapel.h**

Entsprechend müssen in der Implementation **stapel.c** das Attribut **static** für die Variable **spitze** und die Definition der Funktion **empty** gelöscht werden:

```
#include "stapel.h"

/*= Daten =====*/
#define HOEHE 10

static char stapel[HOEHE];
int spitze = 0;

/*= Funktionen =====*/
int full (void) {
    return spitze >= HOEHE;
}

void push (char zeichen) {
    stapel[spitze++] = zeichen;
}

char pop (void) {
    return stapel[--spitze];
}
```

Quelltext 8.9: Stapelverwaltung (modularisiert III): **stapel.c**

Schließlich muss im Rahmenprogramm noch der Aufruf von **empty** durch den Ausdruck **!spitze** ersetzt werden.

Vier Anmerkungen noch dazu:

- Noch einmal: Dies dient nur der Demonstration des Konzepts extern definierter Größen. Tatsächlich beschreibt **spitze** einen inneren Zustand des Stapels. Solche

inneren Zustände sollte man „privat“ halten und Zugriffe darauf nur durch spezielle Funktionen ermöglichen, wie z.B. Lesezugriff durch `empty()`. Durch obige Realisierung wird aber sogar Schreibzugriff von außen ermöglicht – ein fataler Entwurfsfehler! Es fehlen leider die Zeit und der Platz, um an dieser Stelle ein nichttriviales, sinnvolles Beispiel für extern definierte Variablen zu erläutern.

- Unsere Variablenvereinbarungen bislang waren streng genommen Definitionen, wie wir jetzt gesehen haben; auf Deklarationen haben wir bislang ganz verzichtet. Das ist ohne weiteres möglich, auch für Funktionen, weil jede Definition bei Bedarf eine Deklaration impliziert.

Um diese Implikation für Funktionen zu nutzen, müsste man die Reihenfolge ihrer Definitionen ggf. umdrehen, d.h. jede rufende Funktion müsste *hinter* den Funktionen definiert werden, die sie ruft. Das ist in C jedoch nicht üblich und auch nicht immer möglich, wenn sich z.B. Funktionen gegenseitig aufrufen.

- Es sollte selbstverständlich sein, dass innerhalb eines Gesamtprogramms für eine Variable *höchstens* an einer Stelle ein Anfangswert angegeben werden darf. Die Restriktion, dass ein Anfangswert nur in einer Definition zugewiesen werden darf, ist damit naheliegend.
- Wir haben den Begriff *extern* jetzt mit zwei Bedeutungen kennengelernt, bei denen sich allenfalls mit Mühe Gemeinsamkeiten entdecken lassen. Man muss also sauber zwischen *externen* Variablen und Variablen mit dem Speicherklassen-Attribut **extern** unterscheiden.

Wir haben gesehen: Variablen kann man sowohl außerhalb als auch innerhalb von Funktionen deklarieren! Einerseits besagt die Deklaration einer Variablen (gekennzeichnet durch das Attribut **extern**) auf jeden Fall, dass die Definition der Variablen irgendwo anders steht. Andererseits hat das Attribut **extern** keinen Einfluss auf den Gültigkeitsbereich des Namens:

- Eine interne Variable mit dem Attribut **extern** steht nur in dem Block zur Verfügung, der ihre Deklaration enthält; wenn in mehreren Blöcken derselbe Name mit dem Attribut **extern** deklariert ist, stellt der Linker die Verbindung her.
- Eine externe Variable mit dem Attribut **extern** steht von der Stelle ihrer Deklaration an bis zum Ende des Moduls zur Verfügung.

8.7 Statische und automatische Variablen

Eine ähnliche, wenn auch nicht ganz so gravierende Begriffsüberschneidung gibt es bei *statisch/static*.

Eine *statische* Variable steht prinzipiell während der gesamten Dauer der Ausführung eines Programms zur Verfügung, auch wenn man zeitweilig nicht auf sie zugreifen kann, weil sie in einigen Modulen nicht deklariert oder in einigen Funktionen verschattet ist. Statisch sind alle externen Variablen – gleichviel, ob sie das Attribut **static** tragen oder nicht.

Für statische Variablen wird der Speicherplatz bereits durch den Compiler bereitgestellt und zugeordnet. Der Compiler löscht die Speicherbereiche dieser Variablen auch mit binären Nullen, wenn der Programmierer nicht in der Definition explizit Anfangswerte vorgibt. Dass ein Anfangswert neben einer Konstante auch ein konstanter Ausdruck sein darf, sollte klar sein: Ausdrücke, die der Compiler auswerten kann, sind Konstanten gleichgestellt, wie wir bereits gesehen haben.

Das Pendant zu den statischen Variablen sind die *automatischen* Variablen. Sie sind interne Variablen, d.h. Variablen, die innerhalb eines Blocks (zum Beispiel im Rumpf einer

Funktion) definiert werden. Diese Variablen besitzen nur einen Speicherplatz, während der Block ausgeführt wird, in dem sie definiert sind. Oder anders formuliert: Einer automatischen Variablen wird in dem Moment ein Speicherplatz zugeordnet, in dem der Block betreten wird, der ihre Definition enthält; beim Verlassen des Blocks wird die Zuordnung wieder gelöst. Wird der Block erneut betreten, erfolgt eine neue Zuordnung – allerdings unter Umständen zu einem anderen Speicherplatz.

Da der Standard außerdem nicht vorschreibt, dass automatische Variablen bei der Speicherplatzzuordnung einen wohldefinierten Anfangswert erhalten, *muss* man ihnen erst einen Wert zuweisen, bevor man auf ihren Wert zugreifen kann. Diese Zuweisung kann in der Form einer Anfangswertzuweisung erfolgen oder auch durch eine explizite Wertzuweisung; beides löst der Compiler letztlich ohnehin in gleicher Weise auf. Ob wir

```
void beispiel(double a, double b) {  
    double f = sin(a) * exp(b);  
    ...  
}
```

oder

```
void beispiel(double a, double b) {  
    double f;  
    f = sin (a) * exp (b);  
    ...  
}
```

schreiben, ist also gleichgültig. Das erklärt letztlich auch, warum bei der Anfangswertzuweisung für automatische Variablen *beliebige* Ausdrücke erlaubt sind und nicht nur *konstante* Ausdrücke: Da der Compiler den Speicherplatz (noch) nicht bereitstellt, kann er den Wert selbst noch nicht zuweisen, sondern nur eine Befehlsfolge erzeugen, die den Wert berechnet und speichert. Falls diese Befehlsfolge die Aufrufe von Funktionen enthält, kann (und muss) der Linker diese ergänzen. Falls auf Parameter der Funktion zugegriffen wird, ist das auch kein Problem: Die Befehlsfolge wird ja erst ausgeführt, wenn die Funktion aufgerufen wird – und dann sind den Parametern bereits die Argumente des Aufrufs zugeordnet. Übrigens: `sin` und `exp` sind Standardfunktionen, die in der Headerdatei `math.h` deklariert sind. Darauf werden wir im nächsten Abschnitt zurückkommen.

Bei automatischen Variablen darf man das Speicherklassen-Attribut `auto` angeben – aber wer wird das schon tun, da sie dieses Attribut ohnehin automatisch erhalten!

Interessanter ist eine andere Möglichkeit: Variablen, die in einem Block definiert werden, dürfen das Attribut `static` erhalten! Solche Variablen sind eine Mischung aus externen `static`-Variablen und internen Variablen:

- Wie auf interne Variablen kann man auf sie nur in dem Block zugreifen, der ihre Definition enthält. Das impliziert insbesondere, dass sie wie `static`-Variablen lokal sind, also nicht aus anderen Modulen heraus gelesen oder verändert werden können.
- Wie bei externen Variablen erfolgt die Zuordnung des Speicherplatzes einmalig durch den Compiler, was entsprechend auch wieder nur konstante Ausdrücke als Anfangswerte erlaubt.

Wir sehen uns dazu zwei Funktionen an, die bis auf das Attribut `static` für ihre interne Variable übereinstimmen:

```
int f1(void) {  
    int z = 1;  
    return z++;  
}
```

```

}

int f2(void) {
    static int z = 1;
    return z++;
}

```

Rufen wir diese Funktionen durch

```

for (i = 1; i <= 3; i++)
    printf("Aufruf %d: %d - %d\n", i, f1(), f2());

```

auf, so erhalten wir folgende Ausgabe:

```

Aufruf 1: 1 - 1
Aufruf 2: 1 - 2
Aufruf 3: 1 - 3

```

Die Interpretation ist klar:

- Bei **f1** wird der Variablen **z** bei jedem Aufruf erneut ein Speicherplatz und diesem Speicherplatz der Anfangswert 1 zugeordnet, den **f1** auch als Funktionswert liefert. Die Erhöhung des Wertes von **z** als Nebeneffekt bleibt letztlich ohne Wirkung, weil die Zuordnung der Variablen zu dem Speicherplatz mit dem Rücksprung wieder gelöst wird.
- Bei **f2** wird der Variablen **z** bereits durch den Compiler ein Speicherplatz zugeordnet und mit dem Anfangswert 1 belegt. Bei jedem Aufruf liefert **f2** als Funktionswert den Wert, den **z** gerade besitzt – und erhöht als Nebeneffekt den Wert von **z**. Da die Zuordnung zwischen der Variablen **z** und ihrem Speicherplatz mit dem Rücksprung *nicht* gelöst wird, hat **z** beim nächsten Aufruf den veränderten Wert des vorhergehenden Aufrufs.

8.8 register und volatile

Drei Speicherklassen–Attribute haben wir inzwischen kennengelernt, nämlich **auto**, **static** und **extern**. Daneben gibt es zwei weitere Speicherklassen–Attribute, die allerdings nur selten benutzt werden:

- Das Speicherklassen–Attribut **register** zeigt dem Compiler, dass die Variable keine globale Bedeutung besitzt und nach Möglichkeit in einem Register des Prozessors gehalten werden sollte.

Dieses Attribut soll die Laufzeit eines Programms verringern – ob es das wirklich tut, hängt vom jeweiligen Compiler ab:

- Bei einem „guten“ (optimierenden) Compiler und sauberer Programmierung wird das Attribut kaum Wirkung haben, weil der Compiler nach Möglichkeit ohnehin nur mit den Registern arbeiten wird.
- Bei einem „schlechten“ (nicht optimierenden) Compiler kann das Attribut tatsächlich zu einer Beschleunigung führen – es kann aber auch zu einer Verlangsamung führen, wenn man „zu viele“ Variablen mit ihm versieht.

Da der Standard keine detaillierten Vorschriften enthält (und auch nicht enthalten kann), ist man im Einzelfall auf Ausprobieren angewiesen. Vorgeschrieben ist auf jeden Fall: Nur automatische Variablen und Parameter dürfen das Attribut **register** erhalten.

- Das Speicherklassen-Attribut `volatile` verbietet dem Compiler die Optimierung des Zugriffs auf die entsprechend vereinbarten Variablen. Das bedeutet, dass der Wert bei jedem Zugriff neu aus dem Speicher geladen werden muss und nicht z.B. in einem Register vorgehalten werden darf. Erforderlich ist so etwas zum Beispiel dann, wenn man einen Treiber für ein externes Gerät programmiert. Dann kann der Inhalt einer Variablen z.B. durch das Gerät verändert werden und bei einer Optimierung des Zugriffs würde diese Änderung nicht bemerkt. Das geht aber weit über einen Programmierkurs für Anfänger hinaus.

